Problem 1.)

a.)

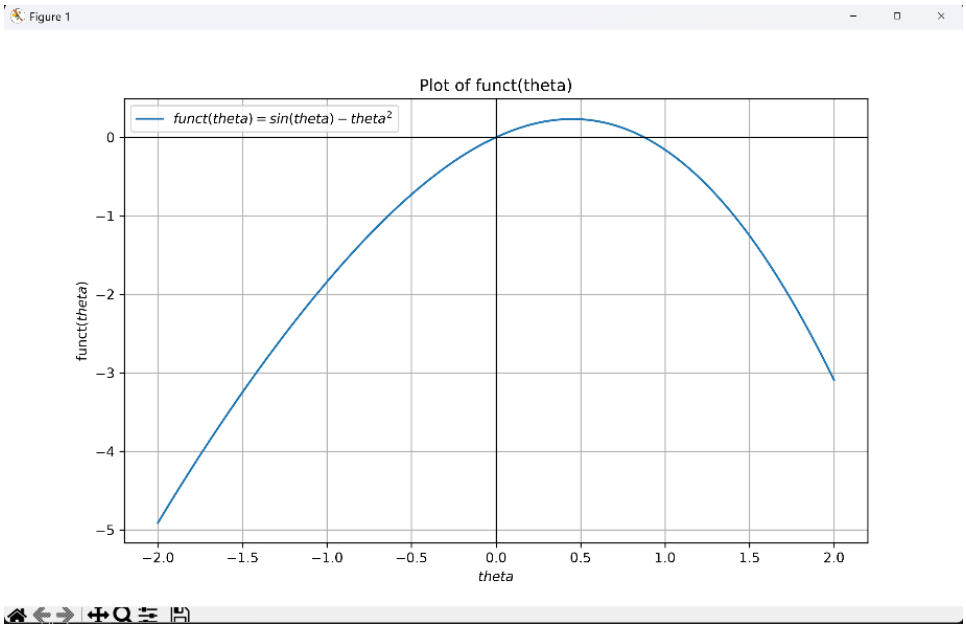


Figure 1.) – Output of sin(theta) – theta^2 from -2 to 2

b.) From the plot, I determine initial bracketing values for root 1 to be [-0.5, 0.5], and for root 2 to be [0.75, 1.25]

c.) From the code, with 100 iterations and the tolerance if 1*10^-8, I found root 1 to be 0.0 and root 2 to be 0.8767262153950625

Code:

```
import numpy as np
import matplotlib.pyplot as plt

def funct(theta): ##Defines the function described in problem 1
    return np.sin(theta) - theta**2

theta = np.linspace(-2, 2, 100) #Start and end of linspace, with # of samples
plt.figure(figsize=(10,6))
plt.plot(theta, funct(theta), label='$sin(theta) - theta^2$') #Actual plot, x, y, and label
plt.axhline(0, color = 'black', linewidth = 1)
plt.axvline(0, color = 'black', linewidth = 1)
plt.grid(True)
```

```
plt.xlabel('$theta$')
plt.ylabel('funct($theta$)')
plt.legend() #Plots label from earlier
plt.title('Plot of $sin(theta) - theta^2$')
plt.show() #Shows plot


def bisection(func, a,  b, tol= (1*(10^-8)), iter=100): #Bisection function def
    for i in range(iter): #Iterates 100 times, (changeable)
        c = (a+b)/2 #Defines c as the halfway point between bracketing values
        if func(a) * func(b) >=0:
            return "Bisection method fails"
        if func(c) == 0 or (b-a) / 2 < tol: #If c is exactly the root or values are within the tolerance, return c immediately
            break
        if func(a) * func(c) < 0: #If the a * b is negative, bring b closer to a
            b = c
        else: #If the a*b is positive, bring a closer to b
            a = c
    return c
root1 = bisection(funct, -.25, .25)
root2 = bisection(funct, .75, 1.25)
print(f"The first root is ~{root1}")
print(f"The second root is ~{root2}")
```

Problem 2.)

a.) Through a similar functionality as problem 1, I got the solution within an error of 10^-4 in 15 iterations.
b.) Through the regula falsi method, I had 6 iterations to find the root to the same accuracy.
c.) It is what I expected. The Regula Falsi method uses the slop of the secant line of the function to better estimate where the root actually is, so I would expect it to work significantly faster than the bisection, that only cuts in half every time.

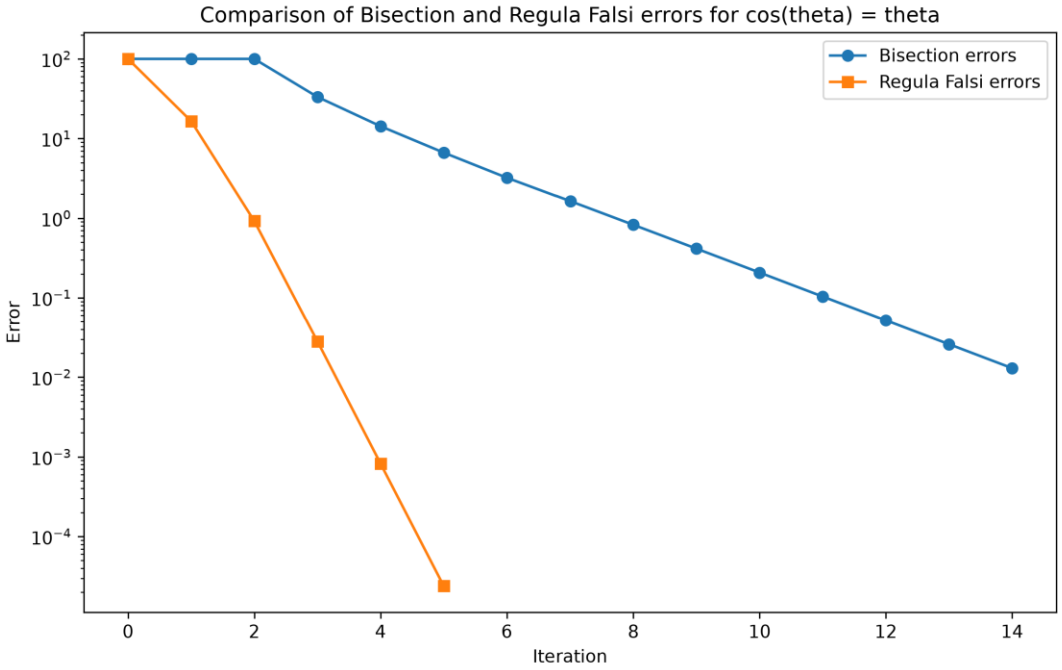


Figure 2.) – Comparison of Bisection errors to Regula Falsi errors

Code

```
import numpy as np

import matplotlib.pyplot as plt

import math


def funct(theta):

    return np.cos(theta) - theta #finding root of cos(theta) = theta can be rewritten as such


def bisection(func, a, b, tol = 1*(10**-4), iter = 100):

    errors = []

    if func(a) * func(b) >= 0:
```

```
        return "Bisection method fails"
    c = a
    iterations = 0
    errors = []
    for i in range(iter):
        c_prev = c
        c = (a + b) / 2
        iterations += 1
        if c!=0:
            err = abs((c - c_prev) / c) * 100
        else:
            err = 0
        errors.append(err)
        if err < tol:
            break
        if abs(func(c)) < tol:
            break
        if abs(c - c_prev) < tol:
            break

        if func(a) * func(c) < 0:
            b = c
        else:
            a = c
    return c, iterations, errors
root_b, iter_b, error_b = bisection(funct, 0, np.pi)
print(f"Bisection root is {root_b}")
print(f"# of Bisection iterations = {iter_b}")

def rf(func, a, b, tol = 1e-4, iter = 100):
    if func(a) * func(b) >= 0:
```

```
        return "Regula Falsi not work"
    errors = []
    c = a
    iterations = 0
    for i in range(iter):
        c_prev = c
        c = (a*func(b) - b* func(a)) / (func(b) - func(a))
        iterations +=1

        if c!= 0:
            err = abs((c - c_prev) / c) * 100
        else:
            err = 0
        errors.append(err)
        if err < tol:
            break
        if func(a) * func(c) < 0:
            b = c
        else:
            a = c
    return c, iterations, errors
root_rf, iter_rf, error_rf = rf(funct, 0, np.pi)
print(f"RF Root is {root_rf}")
print(f"RF # of iterations is {iter_rf}")

plt.figure(figsize = (10, 6))
plt.semilogy(error_b, label = 'Bisection errors', marker = 'o')
plt.semilogy(error_rf, label = "Regula Falsi errors", marker = 's')
plt.xlabel("Iteration")
plt.ylabel("Error")
plt.title("Comparison of Bisection and Regula Falsi errors for cos(theta) = theta")
```

```
plt.legend()
plt.show()
```

Problem 3.)

a.) For sin(x^2) -x to equal 0, x must be sin(x^2) = g(x). My final root was 1.9603451946832443e-10 after 5 iterations.

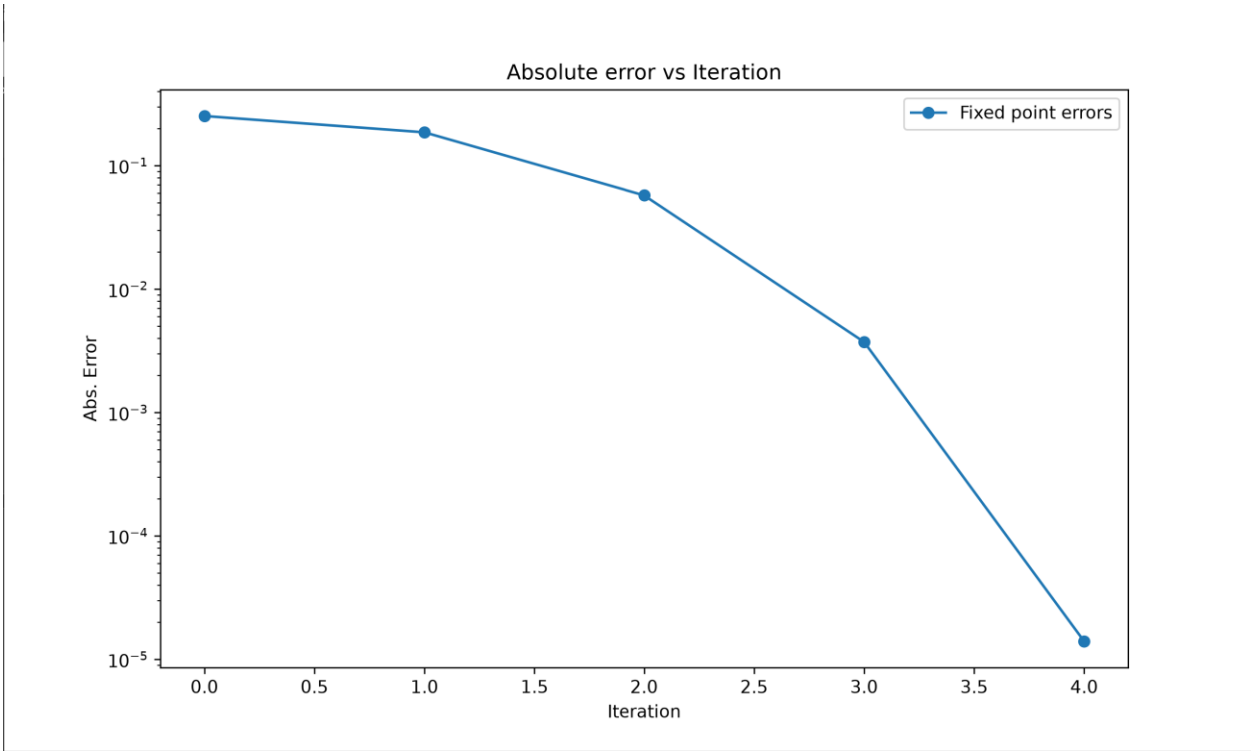


Figure 3.) – Abs error vs Iterations for Fixed Point method

Code

```
import numpy as np
import matplotlib.pyplot as plt

def g(x): #Fixed point function g(x)
    return np.sin(x**2)
def f(x): #original function
    return np.sin(x**2) - x

def fp(func, a, tol=1e-4, iter=100):
    errors = []
    x = a
    for i in range(iter):
```

```
        x_new = func(x)
        err = abs(x_new - x) #calculating abs. error
        errors.append(err)

        if err < tol:
            return x_new, i + 1, errors #If error is within tolerance, print root, iteration number, and errors
        x = x_new
    return x, iter, errors

root_fp, iter_fp, error_fp = fp(g, 0.5)
print(f"Root: {root_fp}")
print(f"Iterations: {iter_fp}")

plt.figure(figsize = (10, 6))
plt.semilogy(error_fp, label = 'Fixed point errors', marker = 'o')
plt.xlabel('Iteration')
plt.ylabel('Abs. Error')
plt.title("Absolute error vs Iteration")
plt.legend()
plt.show()
```

Problem 4.)

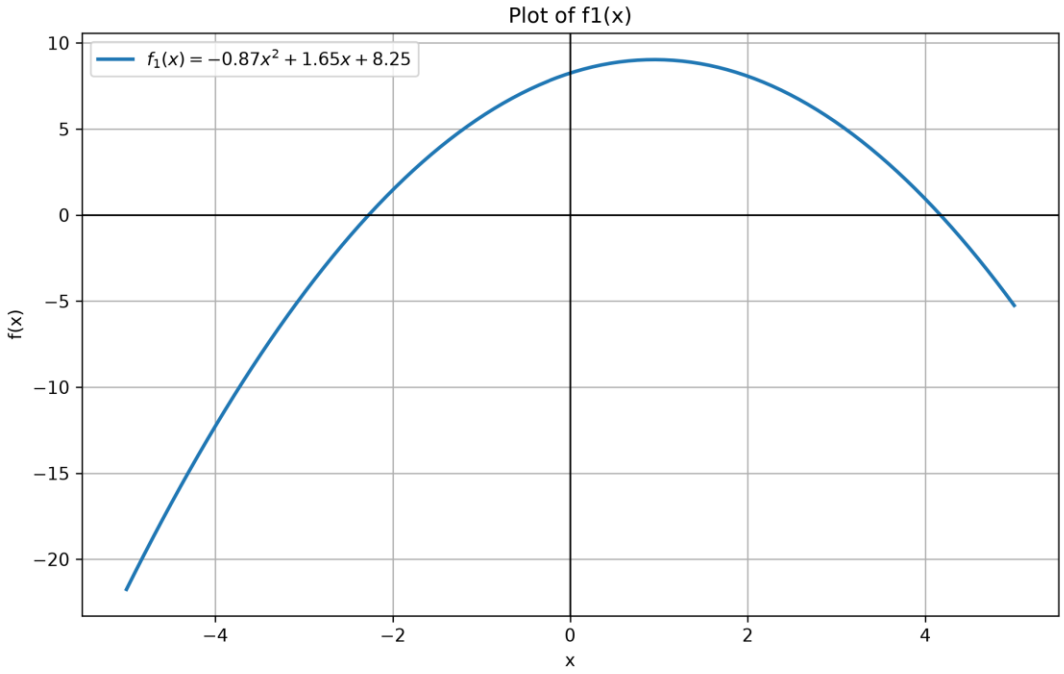


Figure 4.) – f1(x)

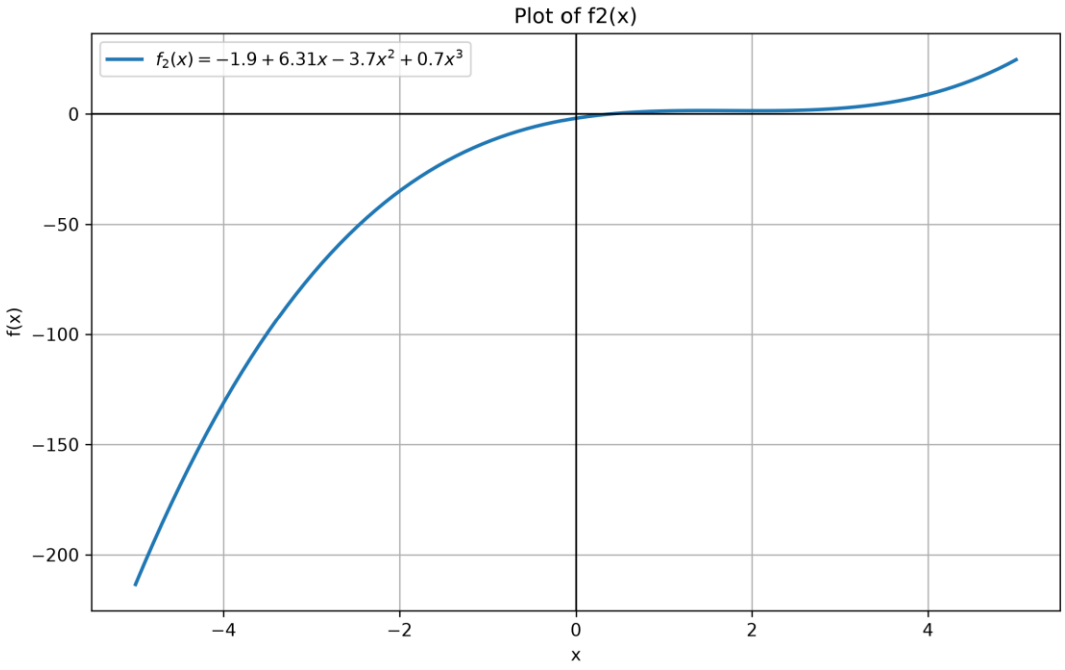


Figure 5.) – f2(x)

a.) From these plots, I estimate the leftmost root of f1(x) to be ~-1.2 and the right root to be ~4.2, and for f2(x) I estimate a root of ~0.4

b.) After implementing newton Raphson, I got the roots of Root1:-2.2738330329363374, Root2=0.37949236247249724. Plugging the solutions back in to the equations, I got values very close to 0, which checks out!

Code:

```
import numpy as np
import matplotlib.pyplot as plt


def f1(x): #function 1
    return -0.87 * x**2 + 1.65 * x + 8.25


def deriv_f1(x): #Derivative of f1
    return -1.74 * x + 1.65


def f2(x):#function 2
    return -1.9 + 6.31 * x -3.7 * x**2 + 0.7 * x**3
def deriv_f2(x):#derivative of function2
    return 6.31 - 7.4 * x + 2.1 * x**2


def nr(func, deriv, a, tol=1e-8, iter=100):
    errors = []
    x = a
    for i in range(iter):
        funct = func(x)
        der = deriv(x)
        if der == 0:
            return "deriv = 0, no solution"
        x_new = x - funct / der #Determining error via NR method
        err = abs(x_new - x) #Calculating absolute error at x
        errors.append(err)
```

```
        if err < tol: #If error is less than tolerance, return answers
            return x_new, i + 1, errors
        x = x_new
    return x, iter, errors

root1, iter1, err1 = nr(f1, deriv_f1, 0)
print(f"Root1:{root1}, iter1={iter1}")
root2, iter2, err2 = nr(f2, deriv_f2, 0)
print(f"Root2={root2}, iter2={iter2}")

# Plot f1(x), (same method as Q1)
x = np.linspace(-5, 5, 100)
plt.figure(figsize=(10, 6))
plt.plot(x, f1(x), label='$f_1(x) = -0.87x^2 + 1.65x + 8.25$', linewidth=2)
plt.axhline(0, color='black', linewidth=1)
plt.axvline(0, color='black', linewidth=1)
plt.grid(True)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Plot of f1(x)')
plt.legend()
plt.show()

# Plot f2(x)
plt.figure(figsize=(10, 6))
plt.plot(x, f2(x), label='$f_2(x) = -1.9 + 6.31x - 3.7x^2 + 0.7x^3$', linewidth=2)
plt.axhline(0, color='black', linewidth=1)
plt.axvline(0, color='black', linewidth=1)
plt.grid(True)
plt.xlabel('x')
plt.ylabel('f(x)')
```

```
plt.title('Plot of f2(x)')
plt.legend()
plt.show()
```

Problem 5.)

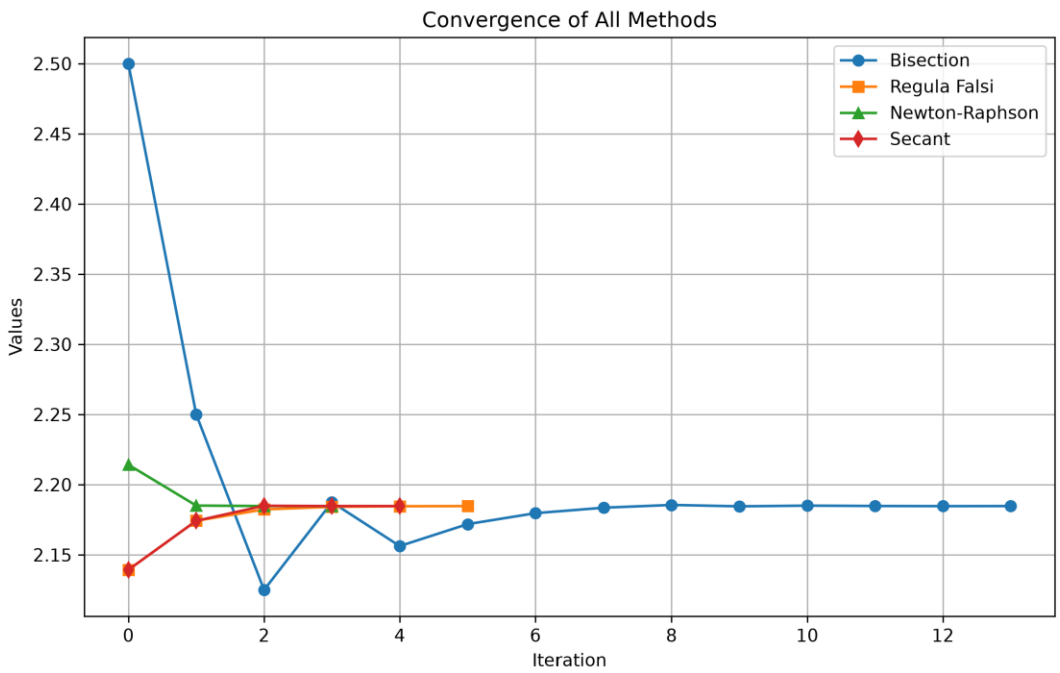


Figure 6.) – Covergence of All methods

All positive solutions are as follows, outputted via my python code:

Bisection Root: 2.18475341796875

Regula Falsi Root: 2.1847283089399587

Newton-Raphson Root: 2.184756114346299

Secant Root: 2.184755216122389

Code:

```
import numpy as np
import matplotlib.pyplot as plt


def f(theta):
    return 3* np.sin(theta) ** 2 + np.cos(theta) - theta **3 + 9


def deriv_f(theta): #deriv for NR
    return 3 * np.sin(2*theta) - np.sin(theta) - 3 * theta**2
```

```
#Bisection method : Same as Q2
def bisect(func, a, b, tol = 1e-4, iter = 100):
    values = []
    if func(a) * func(b) >= 0:
        return "Bisection method fail"
    c = a
    for i in range(iter):
        c_prev = c
        c = (a + b) / 2
        values.append(c)
        if abs(c - c_prev) < tol: #If difference between old & new is less than tol, return root & list
            break
        if func(a) * func(c) < 0:
            b = c
        else:
            a = c
    return c, values
#RF method, same as Q2
def rf(func, a, b, tol = 1e-4, iter = 100):
    values = []
    if func(a) * func(b) >= 0:
        return "RF method no work"
    c = a
    for i in range(iter):
        c_prev = c
        c = (a*func(b) - b* func(a)) / (func(b) - func(a)) #Calclate new C via NR method
        values.append(c)
        if abs(c - c_prev) < tol: #If difference between old & new is less than tol, return root & list
            break
        if func(a) * func(c) < 0:
            b = c
```

```
        else:
            a = c
    return c, values
def nr(func, deriv, a, tol=1e-4, iter = 100):
    values = []
    x = a
    for i in range(iter):
        x_new = x - func(x) / deriv(x) #Determine new x via NR method
        values.append(x_new)
        if abs(x_new - x) < tol: #If difference between old & new is less than tol, return root & list
            break
        x = x_new
    return x, values

def secant(func, a, b, tol = 1e-4, iter = 100):
    values = []
    for i in range(iter):
        f_a = func(a)
        f_b = func(b)
        if f_a == f_b:
            break
        x_new = a - f_a * (b - a) / (f_b - f_a) #Determine new value via Secant method
        values.append(x_new)
        if abs(x_new - b) < tol: #If difference between old & new is less than tol, return root & list
            break
        a = b
        b = x_new
    return b, values

root_bisect, vals_bisect = bisect(f, 2, 3) #Assigning outputs of each function
root_rf, vals_rf = rf(f, 2, 3)
```

```
root_nr, vals_nr = nr(f, deriv_f, 2.5)
root_sec, vals_sec = secant(f, 2, 3)

print(f"Bisection Root: {root_bisect}") #Printing roots for each function
print(f"Regula Falsi Root: {root_rf}")
print(f"Newton-Raphson Root: {root_nr}")
print(f"Secant Root: {root_sec}")

plt.figure(figsize=(10, 6)) #Plotting values at each iteration
plt.plot(vals_bisect, label='Bisection', marker='o')
plt.plot(vals_rf, label='Regula Falsi', marker='s')
plt.plot(vals_nr, label='Newton-Raphson', marker='^')
plt.plot(vals_sec, label='Secant', marker='d')
plt.xlabel('Iteration')
plt.ylabel('Values')
plt.title('Convergence of All Methods')
plt.grid(True)
plt.legend()
plt.show()
```

Problem 6.)

The golden section search gave me better results, likely because the parabolic interpolation only had 5 iterations to work with, while golden section does not. It also ensures the function is bracketed, making it guaranteed to converge.

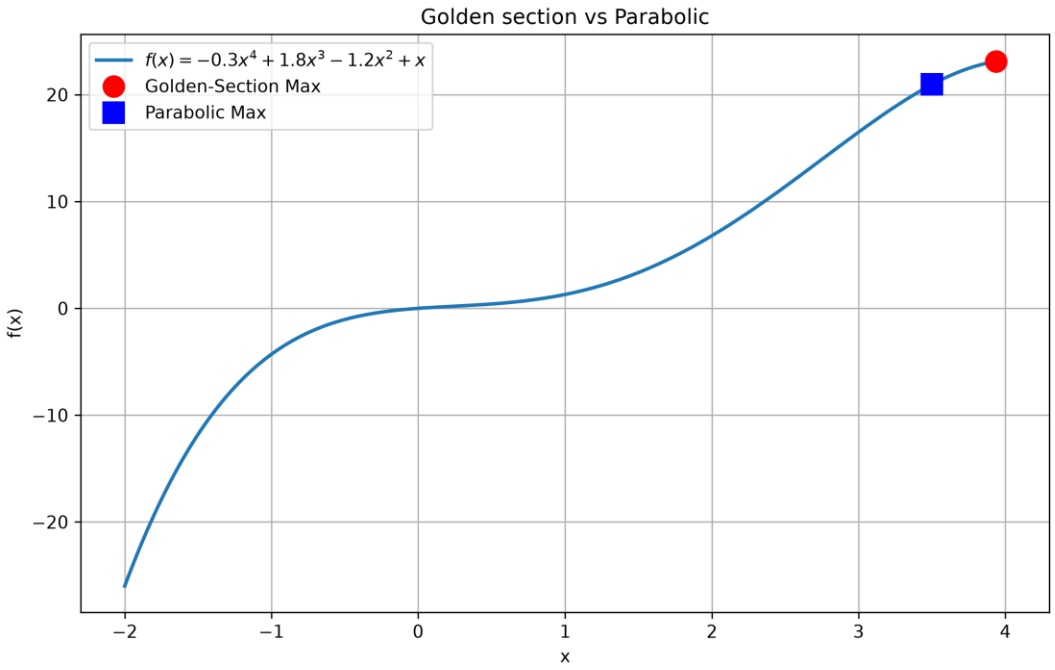


Figure 7.) – Golden vs Parablic maximums

Code

```
import numpy as np

import matplotlib.pyplot as plt


def f(x): #Function

    return -0.3 * x**4 + 1.8 * x**3 - 1.2 * x**2 + x


def neg_f(x): #Minimize f(x) to find maxima

    return -f(x)

def golden(func, xl, xu, es=1, iter = 100):

    phi = (1 + np.sqrt(5)) / 2
```

```
    d = (phi - 1) * (xu - xl) #Interior points for golden section search
    x1 = xu - d
    x2 = xl + d

    iters = 0
    while True:
        iters +=1
        if neg_f(x1) < neg_f(x2): #Compare function values
            xu = x2
        else:
            xl = x1

        d = (phi - 1) * (xu - xl) #Updated Interval
        x1_old = x1
        x1 = xu - d
        x2 = xl + d

        if x2!= 0: #Calculating error
            err = abs((x2 - x1) / x2) * 100
        else:
            err = 0
        if err < es or iters >= iter:
            break
    return (xl + xu) / 2, iters, err

def parabolic(x1, x2, x3, iter = 100):
    for i in range(iter):
        d1 = x2 - x1 #slopes
        d2 = x3 - x2
```

```
    h1 = (neg_f(x2) - neg_f(x1)) / d1
    h2 = (neg_f(x3) - neg_f(x2)) / d2

    denom = (h2 - h1) / (d2 + d1) #calculate denominator

    if denom == 0:
        break
    x_new = x2 - (d1 * h2 - d2 * h1) / (2*denom) #Calculate new estimate for min -f(x)

            # Add this bracketing check
    if x_new < x1 or x_new > x3:
        # If outside, fall back to midpoint
        x_new = x2 + 0.5 * (x3 - x1) * (1 if x2 > (x1+x3)/2 else -1)

    if x_new > x2: #Update points
        if neg_f(x_new) < neg_f(x2):
            x1 = x2
            x2 = x_new
        else:
            x3 = x_new
    else:
        if neg_f(x_new) < neg_f(x2):
            x3 = x2
            x2 = x_new
        else:
            x1 = x_new
return x2, neg_f(x2)
```

```
xl, xu = -2, 4
root_gs, iters_gs, err_gs = golden(neg_f, xl, xu, es=1)
max_val_gs = f(root_gs)

print(f"Golden-Section Search:")
print(f"Root GS: {root_gs:.6f}")
print(f"Max Value: {max_val_gs:.6f}")
print(f"Iterations: {iters_gs}")
print(f"Final Error: {err_gs:.4f}")

x1, x2, x3 = -2, 1, 2
root_pi, min_val_pi = parabolic(x1, x2, x3, iter=5)
max_val_pi = -min_val_pi

print(f"Parabolic Interpolation:")
print(f"Root Parab.: {root_pi:.6f}")
print(f"Max Value: {max_val_pi:.6f}")

x = np.linspace(-2, 4, 400)
plt.figure(figsize=(10, 6))
plt.plot(x, f(x), label='$f(x) = -0.3x^4 + 1.8x^3 - 1.2x^2 + x$', linewidth=2)
plt.plot(root_gs, max_val_gs, 'ro', markersize=12, label=f'Golden-Section Max')
plt.plot(root_pi, max_val_pi, 'bs', markersize=12, label=f'Parabolic Max')
plt.grid(True)
plt.legend()
plt.title('Golden section vs Parabolic')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.show()
```